

第三章 表格型方法

从 **有模型到免模型**：第二章主要探讨了在模型已知（知道所有的概率）的情况下，如何计算得到价值或者最优策略；但当模型未知或过于大无法计算时，则变为免模型：我不知道环境规律，但我可以不断进去试，能不能靠经验学出来？

Q 表格

本质：探索环境学出来的“**经验手册**”。

内容：**记录了所有的 $Q(s, a)$ 的组合（估测值）**。

根据 Q 表，很容易选出在当前状态该做什么动作（选择对应 s 里面 Q 最大的 a ）。

核心问题转化为：如何将这份表格建立起来，且让其数值尽可能准确贴近现实？

免模型预测

蒙特卡洛方法（MC）

上一章讲过，对于固定策略 π ，跑多次记录回报求均值得到 $V_\pi(s)$ 。

此外，大量记录原始回报数据开销大，可以采用 **增量均值** 的方式进行优化：

$$V(s_t) \leftarrow V(s_t) + \alpha [G_t - V(s_t)]$$

也即：**新估计 = 旧估计 + 学习率 × (本次记录值 - 旧估计)**。

蒙特卡洛方法只适用于有限长度的运行（状态转移之间不能成环）。

时序差分（TD）

在 MC 的基础上进行一个小改动：

$$V(s_t) \leftarrow V(s_t) + \alpha [r_{t+1} + \gamma V(s_{t+1}) - V(s_t)]$$

也就是 **把 G_t 换成了 $r_{t+1} + \gamma V(s_{t+1})$** （这样做的合理性见上一章笔记）。

这就是最基本的 **TD(0)**，核心思想：不等完整的 G_t ，而是先用 $r_{t+1} + \gamma V(s_{t+1})$ 估计它。

为什么这么做？MC 有一个问题：想更新 $V(s_t)$ ，需要等到一整个流程全部跑完得到 G_t 。

更快的做法：从 s_t 走一步到 s_{t+1} ，得到了奖励 r_{t+1} ，此时可以根据 $V(s_{t+1})$ 的估计值进行对 $V(s_t)$ 的更新。 $r_{t+1} + \gamma V(s_{t+1})$ 也称为 **TD target**。

TD error (时序差分误差)： 其实就是

$$\delta_t = r_{t+1} + \gamma V(s_{t+1}) - V(s_t)$$

所以 TD 的更新公式可以写成

$$V(s_t) \leftarrow V(s_t) + \alpha \delta_t$$

这里提到一个关键概念：**自举：在更新中是否采用估计值来更新？**

- MC: G_t 是实际记录的奖励值，不是估计值，所以 **没有自举**。
- TD: r_{t+1} 是实际值，但是 $V(s_{t+1})$ 是估计值，所以 **用到了自举**。

n 步 TD

TD 的 **推广：n 步 TD**。TD(0) 实际上就是一步 TD：

$$G_t^{(1)} = r_{t+1} + \gamma V(s_{t+1})$$

推广到一般 (n 步)：

$$G_t^{(n)} = r_{t+1} + \gamma r_{t+2} + \cdots + \gamma^{n-1} r_{t+n} + \gamma^n V(s_{t+n})$$

当 $n \rightarrow \infty$ 时，相当于没有估计了，变成 MC。

也就是说，TD 和 MC 不是两个独立的方法，而是：TD(1步) $\rightarrow$ TD(2步) $\rightarrow \cdots \rightarrow$ 完整回合 = MC。

总结一下这一节，最重要的三条内容：

1. TD 和 MC 的共同目标都是估计 $V_\pi(s)$ 。
2. MC 用完整真实回报 G_t ，而 TD(0) 用一步估计 $r_{t+1} + \gamma V(s_{t+1})$ 。
3. TD 的核心公式：

$$V(s_t) \leftarrow V(s_t) + \alpha \underbrace{[r_{t+1} + \gamma V(s_{t+1}) - V(s_t)]}_{\text{TD error}}$$

免模型控制

上一节是“预测”的问题，这一节要解决“控制”的问题。

回顾一下：预测就是给定策略求某状态的价值；**控制就是求最优策略（价值最大化）**。

对应免模型预测，这里也有两种路径：蒙特卡洛控制和 TD 控制。

蒙特卡洛控制

从 t 时刻状态 S_t 开始出发，形成一条完整的轨迹：

$$S_t, A_t, R_{t+1}, S_{t+1}, A_{t+1}, \dots, S_T$$

记录下

$$G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots$$

然后对 Q 进行更新：

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha (G_t - Q(S_t, A_t))$$

根据更新后的 Q 来更新策略。注意这里需要通过 ε -**贪心** 来更新策略：

以 $1 - \varepsilon$ 的概率选择 Q 最大的动作，但保留 ε 的概率选择其他动作。

为什么需要这样做？因为在免模型的情况下，刚开始得到的 Q 值偏差大，如果从一开始直接选择眼前的最优解可能掉入局部最优解陷阱。

举个例子：第一轮跑完后，计算出 s 状态下 4 种 a 的 Q 表：

a	a_1	a_2	a_3	a_4
$Q(s, a)$	5	3	1	0

如果不采用 ε -贪心，则直接采用 a_1 。但实际上由于一次实验的偏差，真实的模型可能是

$$Q^*(s, a_1) = 4, \quad Q^*(s, a_4) = 10$$

换句话说，如果从一开始永远选择 a_1 ，你就永远没有机会发现 a_4 更好。

实际上， ε 一般是一个逐渐减小的值：随着跑的次数的增多， Q 表收敛到接近实际值的稳定值，可以渐渐减少探索。

$\varepsilon > 0$ **给未知动作留下被探索的机会**，但不保证有限时间内找到全局最优； ε 太小可能过早利用错误的 Q 值，太大又会浪费大量时间进行无效随机探索。好的策略是在学习早期保持较强探索，随着 Q 表越来越可靠逐渐降低 ε ，在“充分探索”和“利用已有知识提高学习效率”之间取得平衡。

TD 控制

回顾一下 TD 预测：核心公式就是

$$V(s_t) \leftarrow V(s_t) + \alpha [r_{t+1} + \gamma V(s_{t+1}) - V(s_t)]$$

只看一步实际奖励，再用下一个状态的估计价值代替更远的未来。

而将其应用于控制，则需要把 **更新的对象从 V 改成 Q** ，也即

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha [r_{t+1} + \gamma Q(s_{t+1}, a_{t+1}) - Q(s_t, a_t)]$$

但这里有一个关键的问题：从 V 改成 Q **多了一个变量：采取的动作 a** 。 t 时刻的动作是已经执行的确定的，但是 $t + 1$ 时刻是未来的动作，我们 **目前不知道会采取什么动作**，所以在这里 TD 控制产生两种分化：Sarsa 和 Q-learning。

Sarsa

a_{t+1} **是通过 ϵ -贪心的当前策略得到的**。一个典型流程：

1. 根据当前 Q 表，用 ϵ -贪心策略在 S_t 选择 A_t ；
2. 执行动作，得到 R_{t+1} 和 S_{t+1} ；
3. 在 S_{t+1} 中，仍然按照同一个 ϵ -贪心策略选择 A_{t+1} ；
4. 用 $R_{t+1} + \gamma Q(S_{t+1}, A_{t+1})$ 更新 $Q(S_t, A_t)$ ；
5. 将 S_{t+1}, A_{t+1} 作为下一步的当前状态和动作。

注意：上面的第四步在完成 Q 的更新后会自动更新下一步的策略（还是 ϵ 贪心，但是会根据新的 Q 最大值对应的动作选）。

Q-learning

不存在实际执行的 a_{t+1} ，而是进行虚拟的更新进入下一轮。一个典型流程：

1. 根据当前 Q 表，用 ϵ -贪心策略在 S_t 选择 A_t ；
2. 执行动作，得到 R_{t+1} 和 S_{t+1} ；
3. 查看 S_{t+1} 下所有动作的 Q 值，**取最大值对应的动作 a'** ；
4. 用 $r_{t+1} + \gamma Q(s_{t+1}, a')$ 更新 $Q(s_t, a_t)$ 。**注意：这里不代表真的执行了 a'** ；
5. $S_t \leftarrow s_{t+1}$, a_t **由更新后的 ϵ 贪心策略产生**。

对比总结

步骤	Sarsa	Q-learning
1	ϵ -greedy 选 A_t	ϵ -greedy 选 A_t
2	执行 A_t , 得到 R_{t+1}, S_{t+1}	相同
3	ϵ -greedy 实际选出 A_{t+1}	找 $\max_a Q(S_{t+1}, a)$, 仅用于学习
4	用 $R + \gamma Q(S', A')$ 更新	用 $R + \gamma \max_a Q(S', a)$ 更新
5	把已选出的 A_{t+1} 作为下一步动作	下一轮再用 ϵ -greedy 选择实际动作

最关键的时序区别其实就在第 3 ~ 5 步：

Sarsa：先实际选 A_{t+1} → 用它更新 → 下一步真的执行它。

而：

Q-learning：先用 $\max Q$ 更新 → 再用 ϵ -greedy 决定实际怎么走。

Sarsa 称为 **同策略时序差分控制**，而 Q-learning 称为 **异策略时序差分控制**，因为

方法	实际行动	TD target 评价未来
Sarsa	ϵ -greedy(Q)	ϵ -greedy(Q)
Q-learning	ϵ -greedy(Q)	greedy(Q)

类似预测中的 TD(n)，控制中 Sarsa 也能拓展到 n 步，但 Q-learning 一般不拓展。